FLEETX

Intelligent Fleet & Logistics Management System

SOFTWARE REQUIREMENTS SPECIFICATION

ARYA COLLEGE OF ENGINEERING & INFORMATION TECHNOLOGY, JAIPUR

PROJECT TEAM

Md Ziaul Mustafa (24EARAD100)

Jitendra Kumar Paswan (24EARAD071)

Mayank Kumar (24EARAD098)

Kushagra Gautam (24EARAD087)

Md Amir Alam (24EARAD099)

PROJECT MENTOR

Ram Babu Buri

Academic Session: 2026–27

Abstract / Executive Summary

FleetX is presented as an intelligent fleet and logistics management web application intended to bring vehicle, driver, trip, shipment, route, tracking, maintenance, fuel, alert, and reporting workflows into one operational view. The stated motivation is that fleet information may otherwise be fragmented across separate records and tools, making it harder to coordinate resources and detect delays, maintenance needs, or route exceptions.

The presentation describes a web architecture with a React 18 and TypeScript frontend, Vite build tooling, Bootstrap 5.3, Leaflet maps using OpenStreetMap tiles, a Three.js visualization, REST communication, and a STOMP client. The backend is described as Java 21 with Spring Boot 3.5.9 and Spring Security, organized as a modular monolith. Persistence is described as MySQL 8.4 using Spring Data JPA, Hibernate, and Flyway migrations. WebSocket/STOMP and SockJS fallback are listed for real-time-style events.

Reported capabilities include session-based authentication, role-based access, CSRF protection, BCrypt password hashing, vehicle and driver records, fuel and maintenance logs, trip dispatch and lifecycle management, shipment manifests, route waypoints, simulated GPS movement, geofences, alerts, and operational reports. The presentation explicitly discloses that telemetry is simulated and that route distance and ETA are estimates based on straight-line Haversine calculations; therefore, the system must not be represented as connected to physical GPS hardware or as providing traffic-aware routing.

This SRS defines a structured set of requirements, interfaces, use cases, data expectations, security considerations, test scenarios, and traceability links. Requirements derived from the presentation are labelled as reported or source-verification-needed where appropriate. Proposed measurable targets are acceptance criteria for future verification, not claims of measured performance. The source ZIP is required to verify exact class names, endpoints, schemas, commands, and whether the automated tests described in the presentation actually execute successfully.

Table of Contents

Preliminary pages 1–4

1. Introduction 5

2. Overall Description 6

3. System Architecture and Design Overview 7–8

4. Functional Requirements 9–12

5. External Interface Requirements 13

6. Non-Functional Requirements 14

7. Use-Case Analysis 15–16

8. Data Requirements and Data Design 17

9. Business Rules, Constraints, and Assumptions 18

10. Security and Privacy Requirements 19

11. Testing and Acceptance Criteria 20–21

12. Requirements Traceability Matrix 22

13. Limitations, Risks, and Future Enhancements 23

14. Deployment and Maintenance 24

15. Conclusion and Appendices 25–26

Pagination is generated by the document renderer and may shift when placeholders are completed or the document is edited. Update the Word table of contents/page references after final source verification.

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the intended capabilities, interfaces, constraints, quality requirements, and verification approach for FleetX. It provides a shared reference for the student team, mentor, tester, and maintainer. It is intended to reduce ambiguity about what the system supports and how completion should be assessed.

1.2 Scope

FleetX is presented as a web-based fleet and logistics management application. Its scope includes centralized operational records for vehicles and drivers; trip planning and dispatch; shipment manifests; route waypoints and timelines; fuel and maintenance records; map-based tracking; simulated GPS updates; geofence-related alerts; role-aware access; and reports. Exact boundaries and feature completeness require source-code inspection.

1.3 Background and Motivation

Fleet operations involve connected entities: vehicles, drivers, trips, shipments, routes, fuel consumption, and maintenance. When these records are kept in disconnected tools, operators may have to reconcile information manually. A unified application can provide a common operational view, support traceability, and make the status of planned and in-progress work easier to review.

1.4 Problem Statement

The project addresses fragmented fleet and logistics information and the difficulty of coordinating vehicles, drivers, trips, and shipments through a shared view. Manual status checking can delay awareness of exceptions. FleetX aims to consolidate records and provide operational visibility; the presentation does not provide measured evidence of business impact, so benefits are objectives rather than proven outcomes.

1.5 Objectives

- Centralize fleet, driver, trip, shipment, fuel, and maintenance information.
- Present trip progress and route waypoints in a map-oriented interface.
- Support operational decisions with alerts, fuel-efficiency metrics, maintenance information, and reports.
- Protect access using session-based authentication, role-based authorization, and CSRF protection, as reported in the presentation.
- Clearly label simulated telemetry and estimated route calculations.

1.6 Existing Approach and Limitations

The presentation describes a general context in which fleet data may be fragmented across records and tools. It does not identify a named legacy system or provide a comparative evaluation. Therefore, this SRS treats fragmentation and manual status checking as the motivating problem, not as a measured audit of a specific existing product.

1.7 Definitions and Abbreviations

Term	Meaning
SRS	Software Requirements Specification
REST	Representational State Transfer
API	Application Programming Interface
CSRF	Cross-Site Request Forgery
JPA	Java Persistence API
ORM	Object-Relational Mapping
ETA	Estimated Time of Arrival
GPS	Global Positioning System
STOMP	Simple Text Oriented Messaging Protocol
SPA	Single-Page Application
RBAC	Role-Based Access Control
E2E	End-to-End
PBL	Project-Based Learning

1.8 Intended Audience and Document Organization

The document is written for an undergraduate engineering mentor and project team. Chapters define product context, architecture, functional and quality requirements, interfaces, use cases, data expectations, security, testing, traceability, limitations, deployment, and appendices. The source ZIP should be used to confirm all implementation-level details before approval.

2. Overall Description

2.1 Product Perspective

The presentation depicts FleetX as a single deployable modular monolith with a separate web frontend and relational database. It explicitly states that the backend is not a microservices system. The browser communicates with the backend through REST and WebSocket/STOMP-style events, while the backend uses JPA/Hibernate and Flyway to interact with MySQL.

2.2 Product Functions

- Identity and administration: login, sessions, roles, and user management.
- Fleet and assets: vehicle directory, fuel logs, and maintenance records.
- Drivers and trips: driver onboarding, dispatch, and trip lifecycle.
- Shipments and routes: shipment manifests, waypoints, and timelines.
- Tracking and control tower: simulated GPS movement, zones/geofences, alerts, and reports.

2.3 User Classes

Role reported in presentation	Expected responsibilities	Access boundary
Admin	Manage users and system-level access	Administrative functions; exact permission matrix requires verification
Fleet Manager	Oversee vehicles, assets, and maintenance	Fleet and maintenance operations
Dispatcher	Coordinate trips, drivers, and shipment movement	Trip and shipment operations
Driver	Access relevant trip and shipment information	Restricted operational view
Viewer	View operational information	Read-only or restricted access as implemented; verify exact permissions

2.4 Operating Environment

Reported technologies are Java 21, Spring Boot 3.5.9, Spring Security, MySQL 8.4 LTS, Spring Data JPA, Hibernate, Flyway, React 18, TypeScript, Vite, Bootstrap 5.3, Leaflet, OpenStreetMap tiles, Three.js, WebSocket/STOMP, SockJS fallback, Maven, and npm. Browser versions, operating-system support, container definitions, environment variables, and deployment topology are not verifiable from the presentation alone.

2.5 Hardware and Software Requirements

A development or deployment environment must support the Java runtime/build toolchain, Node.js/npm frontend toolchain, MySQL service, and a modern browser. Exact minimum RAM, CPU, disk, browser versions, and production sizing are not specified in the supplied presentation and must be determined through testing.

2.6 Constraints, Assumptions, and Dependencies

- The architecture is described as a modular monolith, not microservices.
- Location updates are simulated and labelled “SIMULATED DATA”.
- Distance and ETA are estimates based on Haversine straight-line calculations, not road-network routing or traffic prediction.
- The presentation reports a persistent Docker volume for local MySQL storage; actual Compose/service configuration requires source verification.
- External map tiles depend on the configured OpenStreetMap tile service and network access.
- The exact role-to-permission matrix, endpoint catalogue, schema, and configuration keys are not included in the presentation.

2.7 User Documentation

The final project should include setup instructions, environment configuration, login and role descriptions, common workflows, simulation disclosures, troubleshooting, and test commands. The presentation states that README.md documents commands; the exact content must be checked against the repository.

3. System Architecture and Design Overview

3.1 High-Level Architecture

The presentation identifies three principal tiers: a React/TypeScript browser frontend, a Java/Spring Boot modular-monolith backend, and a MySQL database. The frontend includes a web UI, Leaflet maps, a Three.js view, a REST client, and a STOMP client. The backend is described as containing Auth, Fleet, Drivers & Trips, Shipments, and Tracking modules. Persistence uses JPA/Hibernate and schema migrations use Flyway.

```
SYSTEM CONTEXT
[Admin / Fleet Manager / Dispatcher / Driver / Viewer]
                    |
                    v
          [FleetX Web Frontend]
           | REST       | WebSocket/STOMP
           v            ^
       [FleetX Spring Boot Backend]
                    |
                    v
                [MySQL]

External dependency: OpenStreetMap map tiles (browser-side display)
Note: GPS telemetry is simulated; physical GPS hardware is not established.
```

3.2 Component Responsibilities

Component / module	Responsibility described or implied by presentation	Verification note
Frontend SPA	Operational screens, forms, dashboards, maps, Three.js view	Exact routes/components require source inspection
Authentication / administration	Login, sessions, role-aware user management	Verify security configuration and permission annotations
Fleet / assets	Vehicle directory, fuel records, maintenance	Verify entities, validation, and endpoints
Drivers and trips	Driver onboarding, dispatch, trip lifecycle	Verify state transitions and relationships
Shipments / routes	Manifest details, waypoints, shipment timeline	Verify persistence and calculation logic
Tracking / control tower	Simulated coordinates, map updates, zones, alerts, reports	Verify scheduler, event payloads, and alert rules
Persistence layer	MySQL via Spring Data JPA/Hibernate; Flyway migrations	Verify migrations and entity mappings

3.3 Execution Lifecycle and Request Processing

At a high level, a user opens the frontend, authenticates, and performs an authorized operational action. The frontend submits a REST request or subscribes to real-time-style events. The backend applies authentication/authorization and business processing, reads or updates persisted records through the persistence layer, and returns a response or publishes an event. This flow is a logical description based on the presentation; exact filters, controllers, services, transaction boundaries, and event destinations require source verification.

3.4 Tracking and Simulation

The presentation names SimulatedGpsService and states that coordinates are interpolated along route waypoints. It also says STOMP WebSocket events update tracking views and operational state. The implementation must make simulated data visibly distinguishable from real telemetry. Haversine calculations estimate straight-line distance; any displayed ETA must be labelled as an estimate and must not imply traffic-aware or road-network accuracy.

3.5 Error Handling

The application should return understandable validation and operational errors without leaking stack traces, credentials, or sensitive configuration. The presentation does not disclose exact error DTOs, HTTP status mapping, logging structure, or retry behavior. Those details remain source-verification items.

```
SEQUENCE: REPRESENTATIVE TRIP UPDATE (LOGICAL)
User -> Frontend: submit trip update
Frontend -> Backend: authenticated REST request + CSRF token
Backend -> Security: validate session, role, CSRF
Security -> Trip module: allow authorized request
Trip module -> Persistence: validate and save trip state
Persistence --> Trip module: persisted result
Trip module --> Frontend: response / error
Tracking module --> Frontend: STOMP event (if emitted)
This is a logical sequence; exact endpoint and event names require source verification.
```

4. Functional Requirements

Requirement status convention: “Reported—verify in source” means the presentation states the feature, but the ZIP was not available for code-level verification. “Proposed” means the item is a recommended requirement or acceptance condition rather than a confirmed implementation. Priorities are proposed for project evaluation.

ID / name	Requirement statement	Priority	Status / evidence
FR-001 Authentication	The system shall allow an eligible user to sign in using the configured authentication flow.	Must	Reported—verify in source
FR-002 Session handling	The system shall maintain and terminate authenticated sessions according to the configured security policy.	Must	Reported—verify in source
FR-003 Role authorization	The backend shall enforce permissions for the configured roles; hiding a UI control alone shall not grant security.	Must	Reported—verify exact matrix
FR-004 User administration	The system shall provide administrative user-management functions to authorized administrators.	Should	Reported—verify in source
FR-005 Vehicle directory	The system shall store and present vehicle identification, type, capacity, and operational status.	Must	Reported—verify in source
FR-006 Driver records	The system shall support driver onboarding and relevant licence details.	Must	Reported—verify validation fields
FR-007 Fuel records	The system shall record fuel-log information and calculate fuel-efficiency metrics such as kilometres per litre where inputs permit.	Should	Reported—verify formula and units
FR-008 Maintenance	The system shall maintain vehicle maintenance records and present upcoming/preventive service information where supported by stored data.	Should	Reported—verify scheduling rules
FR-009 Trip lifecycle	The system shall support planned and in-progress trip operations.	Must	Reported—verify allowed states
FR-010 Dispatch	The system shall allow an authorized dispatcher to coordinate trips with relevant drivers, vehicles, and shipment records.	Must	Reported—verify assignment rules
FR-011 Shipment manifest	The system shall maintain shipment code, sender/receiver, weight, volume, and cargo description where defined by the implementation.	Must	Reported—verify field names/types
FR-012 Route waypoints	The system shall represent planned route points and display them in the relevant route/timeline views.	Must	Reported—verify ordering rules
FR-013 Simulated tracking	The system shall generate simulated coordinate updates along route waypoints and label the resulting telemetry as simulated.	Must	Reported—verify service behaviour
FR-014 Map display	The system shall display route waypoints and available vehicle/trip positions on a Leaflet map using configured map tiles.	Should	Reported—verify UI
FR-015 Real-time events	The system shall deliver configured tracking/operational events to subscribed clients through WebSocket/STOMP where enabled.	Should	Reported—verify destinations and payloads
FR-016 Zones and alerts	The system shall present supported zones/geofence-related alerts to authorized users.	Should	Reported—verify exact trigger rules
FR-017 Distance and ETA	The system shall identify distance/ETA values derived from straight-line Haversine calculations as estimates.	Must	Reported explicitly in presentation
FR-018 Reports	The system shall provide the operational reports implemented by the application to authorized users.	Could	Reported—report catalogue needs verification
FR-019 CSRF	State-changing requests shall include and validate the CSRF token as configured by the application.	Must	Reported—verify configuration

ID / name	Requirement statement	Priority	Status / evidence
FR-020 Password storage	Passwords shall be stored using the reported BCrypt password-hashing mechanism, not as plaintext.	Must	Reported—verify encoder and stored values
FR-021 Schema migration	Database schema changes shall be managed through Flyway migrations where configured.	Must	Reported—verify migration files
FR-022 Input validation	The system should validate required fields, types, ranges, and domain constraints before committing changes.	Must	Proposed pending source review
FR-023 Safe errors	The system should return consistent, non-sensitive error responses for invalid input and failed operations.	Should	Proposed pending source review
FR-024 Auditability	The system should retain enough operational history to explain important trip and shipment state changes, if this is within the project scope.	Could	Proposed; persistence/logging to verify

4.1 Requirement Detail and Acceptance Criteria

For each requirement, acceptance should be demonstrated with a test tied to its ID. At minimum, test successful and unsuccessful authorization paths; valid and invalid data; expected state changes; persistence after application restart where persistent storage is intended; and consistency between REST responses and any event-driven updates.

Requirement	Precondition / trigger	Expected postcondition and acceptance
FR-001	A configured user submits valid credentials	Authentication succeeds and the application establishes the configured session; invalid credentials are rejected without disclosing whether a password is correct.
FR-003	A user attempts an action with a role lacking permission	Backend rejects the request; no unauthorized state change occurs.
FR-005	Authorized user submits a valid vehicle record	Record is saved and appears in the vehicle directory; invalid required fields are rejected.
FR-009/010	Authorized dispatcher creates or updates a trip	Only allowed lifecycle transitions are accepted; assignment references are validated.
FR-011/012	User saves a shipment and its waypoints	Manifest values and waypoint order are retained and displayed consistently.
FR-013/017	Tracking simulation is started or viewed	Coordinates follow configured waypoint progression; UI labels data as simulated and distance/ETA as estimates.
FR-019	Client sends a state-changing request without valid CSRF token	Request is rejected according to security configuration and no mutation is committed.
FR-021	A clean database is initialized	Migrations apply in order and application schema validation succeeds, if that is the configured startup process.

5. External Interface Requirements

5.1 User Interface

The presentation describes an operational web UI with dashboards/control-tower views, Leaflet map display, a Three.js view, fleet and driver records, trip and shipment workflows, and maintenance/fuel information. Exact page names, navigation routes, field labels, accessibility semantics, and responsive breakpoints are not shown in the presentation and must be verified in frontend source files.

The UI should make the current state, validation failures, loading state, and permission restrictions understandable. Simulated GPS information should be labelled clearly. Distance and ETA should include an indication that they are estimates. Interactive maps should not be the only way to access essential trip or shipment information.

5.2 Software Interfaces

Interface	Purpose	Evidence / caveat
Browser to backend REST	Create/read/update operational records and authenticate	REST client reported; endpoint list unavailable
Browser to backend WebSocket/STOMP	Receive tracking and operational events	STOMP client and SockJS fallback reported; exact topics unavailable
Backend to MySQL	Persistent application data	MySQL 8.4, JPA/Hibernate reported
Flyway to database	Apply versioned schema migrations	Reported; migration filenames unavailable
Browser to map tile service	Render map tiles	Leaflet and OpenStreetMap tiles reported; exact URL/configuration unavailable

5.3 API Requirements

The supplied presentation confirms REST communication but does not list actual HTTP methods, endpoint paths, request schemas, response fields, status codes, or error bodies. It would be unsafe to invent an API catalogue. The source ZIP must be inspected for controller mappings, DTOs, security rules, and frontend service calls before this section can be converted into an exact endpoint reference.

API catalogue field	Required source evidence
HTTP method and path	Controller mapping annotations or router configuration
Purpose and actor	Controller/service logic and frontend call sites
Request parameters/body	Request DTOs, validation annotations, or parameter declarations
Response schema	Return DTOs, response wrappers, serialization configuration
Status codes and errors	Exception handlers, response annotations, tests
Authentication/CSRF	Spring Security configuration and endpoint-specific rules

5.4 Hardware and Communication Interfaces

No physical GPS receiver, vehicle telematics unit, fuel sensor, or other hardware interface is established by the presentation. Tracking is explicitly described as simulated. Network communication is expected between browser, backend, database, and map tile service. Production firewall rules, TLS termination, hostnames, ports, and deployment topology are not specified.

6. Non-Functional Requirements

The targets below are proposed acceptance targets for planning and verification unless explicitly marked as reported design properties. They are not measured results. Test environment, data volume, browser, database configuration, and load profile must be recorded when evaluating each target.

ID	Requirement	Proposed verification target	Status
NFR-PER-001	Standard read requests should complete within a defined response-time budget under normal test load.	95th percentile ≤ 2 seconds for agreed test endpoints at 25 concurrent users; exclude external tile latency.	Proposed
NFR-PER-002	Write operations should provide timely feedback.	95th percentile ≤ 3 seconds for agreed CRUD operations under the same workload.	Proposed
NFR-REL-001	The application should handle invalid requests without crashing the service.	Run boundary/negative tests; no unhandled process termination.	Proposed
NFR-REL-002	Persisted records should remain available after a normal application restart.	Create, restart, and verify records against configured persistent database.	Proposed
NFR-SEC-001	Protected backend operations shall enforce authorization independent of UI visibility.	All role/action matrix tests produce expected allow/deny outcomes.	Must; implementation to verify
NFR-SEC-002	State-changing requests shall be protected against CSRF according to the configured session model.	Missing/invalid CSRF token is rejected for protected mutations.	Reported design; verify
NFR-SEC-003	Passwords shall not be stored in plaintext.	Verify BCrypt encoder/configuration and persisted representation in a safe test database.	Reported design; verify
NFR-USA-001	Core workflows should be understandable to a first-time student evaluator.	At least 4 of 5 representative evaluators complete assigned tasks without assistance; proposed usability test.	Proposed
NFR-ACC-001	Essential form controls should have programmatic labels and keyboard access.	Manual keyboard and accessibility inspection of core workflows.	Proposed
NFR-MNT-001	Modules should be separable by responsibility within the modular monolith.	Review package/module boundaries and dependency direction.	Reported architecture; verify
NFR-COMP-001	Frontend production build and backend test/build should run using documented repository commands.	Clean checkout build succeeds in documented environment.	Presentation lists commands; verify
NFR-DATA-001	Invalid domain data should not be persisted.	Negative tests verify validation and database constraints.	Proposed
NFR-OBS-001	Operational errors should be diagnosable without exposing secrets to users.	Review logs and client errors for useful context and sensitive-data exclusion.	Proposed
NFR-PORT-001	Development setup should be reproducible on a supported environment.	Second clean environment follows README without undocumented manual steps.	Proposed

6.1 Performance Measurement Protocol

Performance results should report hardware/OS, Java and Node versions, database configuration, sample dataset size, concurrency, test duration, warm-up, and percentile response times. Browser map-tile download time should be separated from backend response time. A target is not a result: measured values should be recorded only after tests are executed.

7. Use-Case Analysis

7.1 Actors

Actor	Goal	Access summary
Admin	Administer users and system access	Administrative role as configured
Fleet Manager	Maintain vehicle, fuel, and maintenance records	Fleet operations
Dispatcher	Plan trips and coordinate shipments/drivers	Dispatch operations
Driver	View relevant trip and shipment information	Restricted operational access
Viewer	Review operational information	Restricted viewing access

```
USE-CASE OVERVIEW (ROLE/FEATURE VIEW)
Admin -------- (Manage users and access)
Fleet Manager - (Manage vehicles / fuel / maintenance)
Dispatcher ---- (Manage trips / dispatch / shipments / waypoints)
Driver -------- (View assigned trip and shipment details)
Viewer -------- (View permitted operational dashboard/reports)
Authorized users (as configured) -- (View tracking / alerts)
Note: Exact role-to-use-case authorization must be verified against backend security rules.
```

7.2 UC-01: Sign In

Primary actor: any configured user. Preconditions: the user account exists and the service is available. Trigger: the user submits login credentials. Main flow: (1) open login screen; (2) enter credentials; (3) submit; (4) backend validates credentials; (5) a valid session is established and the user reaches an authorized view. Alternative flow: invalid credentials produce a safe failure message. Exception flow: service/database failure is handled without revealing secrets. Postcondition: a valid session exists only after successful authentication. Related requirements: FR-001, FR-002, FR-019, NFR-SEC-001.

7.3 UC-02: Maintain Vehicle Record

Primary actor: Fleet Manager or another authorized role. Preconditions: authenticated session and permission to manage fleet assets. Trigger: create or edit a vehicle. Main flow: enter identification, type, capacity, and status; submit; validate; save; show updated record. Alternative flow: invalid or duplicate data is rejected according to domain rules. Exception flow: persistence failure leaves the user informed and avoids reporting a false success. Postcondition: valid vehicle data is persisted. Related requirements: FR-005, FR-022.

7.4 UC-03: Plan and Dispatch Trip

Primary actor: Dispatcher. Preconditions: relevant vehicle and driver records exist; user has dispatch permission. Trigger: create or update a trip. Main flow: enter trip details, associate relevant resources and shipment information, define route waypoints, and save. If the project supports dispatch state transitions, the dispatcher initiates the permitted transition. Alternative flow: missing or conflicting assignments are rejected. Exception flow: failed save or unavailable referenced record is reported. Postcondition: trip state and relationships reflect the accepted operation. Related requirements: FR-009, FR-010, FR-011, FR-012.

7.5 UC-04: Monitor Simulated Vehicle Movement

Primary actor: authorized operational user. Preconditions: a relevant trip/route exists and tracking simulation is available. Trigger: open tracking/control-tower view or start a supported simulation. Main flow: display route waypoints; receive simulated coordinate updates; update map and operational view; show supported alerts. Alternative flow: no active simulation or route data produces a clear empty state. Exception flow: WebSocket disconnect or malformed event is handled gracefully. Postcondition: the user sees the latest available simulated position and any supported operational alerts. Related requirements: FR-013, FR-014, FR-015, FR-016, FR-017.

7.6 UC-05: Review Fuel and Maintenance

Primary actor: Fleet Manager. Preconditions: authenticated user with relevant access. Trigger: open fuel or maintenance section. Main flow: review records; add or update an entry if permitted; inspect efficiency/service information. Alternative flow: incomplete fuel values prevent a misleading efficiency calculation. Postcondition: valid records are saved and displayed. Related requirements: FR-007, FR-008.

7.7 Use-Case to Requirement Mapping

Use case	Related requirements
UC-01 Sign In	FR-001, FR-002, FR-003, FR-019
UC-02 Maintain Vehicle	FR-005, FR-022, FR-023
UC-03 Plan/Dispatch Trip	FR-009, FR-010, FR-011, FR-012
UC-04 Monitor Tracking	FR-013, FR-014, FR-015, FR-016, FR-017
UC-05 Fuel/Maintenance	FR-007, FR-008
Cross-cutting	FR-020, FR-021, NFR-SEC-001, NFR-DATA-001

8. Data Requirements and Data Design

8.1 Data Entities Identified from the Presentation

The presentation implies the following conceptual entities: User/Role, Vehicle, Driver, Trip, Shipment, Route Waypoint, Fuel Log, Maintenance Record, Tracking Position/Event, Zone/Geofence, Alert, and Report. These are conceptual labels derived from feature descriptions, not verified database table or Java entity names.

Conceptual entity	Fields mentioned or implied	Verification needed
User / Role	Credentials/account identity, assigned role	Actual user schema, role representation, account status
Vehicle	Identification, type, capacity, operational status	Field names, units, uniqueness, relationships
Driver	Identity/onboarding and licence details	PII fields, licence expiry validation, assignment rules
Trip	Lifecycle state and resource associations	Actual state enum, start/end fields, vehicle/driver/shipment links
Shipment	Code, sender, receiver, weight, volume, cargo description	Field types, units, required/optional fields
Waypoint	Route point coordinates and order	Latitude/longitude bounds, ordering, trip association
Fuel Log	Fuel record and efficiency inputs	Units, odometer validation, division-by-zero handling
Maintenance Record	Service information and upcoming/preventive needs	Due-date logic, status values, reminders
Tracking event	Simulated coordinates and related route/trip	Event schema, timestamps, retention
Zone / Alert	Zones/geofence and alert information	Polygon/radius representation, trigger and acknowledgement rules

8.2 Persistence and Lifecycle

The presentation states that MySQL 8.4 is used with Spring Data JPA/Hibernate and Flyway migrations, and that a persistent Docker volume is used for local database storage. Exact tables, foreign keys, cascade rules, indexes, transaction boundaries, retention policies, and backup arrangements are not supplied. These must be documented from migrations, entity mappings, configuration, and deployment files.

8.3 Data Validation and Integrity

- Coordinates should be validated to legal latitude and longitude ranges where coordinate inputs are accepted.
- Positive physical quantities such as shipment weight/volume and vehicle capacity should be constrained according to business rules.
- Fuel-efficiency calculations must define units and handle missing, zero, or inconsistent odometer/fuel values.
- Trip and shipment references should point to valid records; allowed lifecycle transitions should be defined explicitly.
- Dates and timestamps should use a consistent timezone policy; the policy is not specified in the presentation.
- Sensitive account data should be minimized and never included in client error messages or logs unnecessarily.

8.4 Data Model Diagram

```
CONCEPTUAL DATA MODEL — NOT A VERIFIED DATABASE ERD
User -- assigned Role
Vehicle --< Trip >-- Driver
Trip --< Shipment association / shipment records
Trip --< Route Waypoint
Vehicle --< Fuel Log
Vehicle --< Maintenance Record
Trip --< Tracking Event
Zone --< Alert

Relationship cardinalities, join entities, and actual table names must be confirmed from source code and Flyway migration
```

9. Business Rules, Constraints, and Assumptions

ID	Rule / assumption	Classification
BR-001	The application is presented as a single deployable modular monolith rather than a microservices system.	Reported architecture
BR-002	Tracking telemetry is simulated and must be labelled as simulated.	Reported and explicit
BR-003	Distance and ETA are based on straight-line Haversine calculation and are estimates.	Reported and explicit
BR-004	Backend authorization, not UI visibility alone, is the security boundary.	Reported security principle
BR-005	State-changing requests require CSRF token forwarding according to the described security setup.	Reported; verify exact exceptions
BR-006	Database schema evolution is managed with Flyway migrations.	Reported; verify migration setup
BR-007	The exact permission matrix for Admin, Fleet Manager, Dispatcher, Driver, and Viewer must be taken from backend security configuration.	Verification requirement
BR-008	Production hosting, TLS termination, secrets management, backup schedule, and monitoring are environment-specific and not specified in the presentation.	Not specified
BR-009	No physical GPS hardware integration is established by the supplied artifact.	Scope boundary based on presentation
BR-010	Performance values in this SRS are proposed acceptance targets, not measured results.	Document rule

9.1 Technical Constraints

The stated stack constrains development to compatible Java/Spring, Node/npm, MySQL, and browser environments. The use of a modular monolith reduces deployment complexity compared with independently deployed services but still requires clear module boundaries. Map rendering depends on tile availability. Simulated telemetry cannot be used as evidence of actual vehicle location.

9.2 Assumptions Requiring Confirmation

- The source repository contains the frontend and backend corresponding to the presentation.
- The commands listed in the presentation README remain accurate for the supplied revision.
- The local Docker volume is configured in deployment files and is not merely a presentation statement.
- The described automated tests are present and runnable; counts in the presentation are not treated as independently verified.
- All five role labels correspond to real application roles; exact permissions are source-defined.

10. Security and Privacy Requirements

10.1 Reported Security Controls

The presentation reports Spring Security server-side sessions, role-based authorization, CSRF protection for state-changing requests, BCrypt password hashing, and secure session cookies. These are treated as reported controls, not independently verified. The project source is needed to confirm session cookie flags, session fixation settings, CSRF exclusions, password encoder configuration, authorization matchers, and account-management behavior.

10.2 Security Requirements

ID	Requirement	Verification
SEC-001	The backend shall authorize each protected operation using the authenticated principal and configured role/permission rules.	Attempt each protected endpoint as every role, including direct requests that bypass UI controls.
SEC-002	The application shall reject invalid or missing CSRF tokens for protected state-changing requests, subject to documented framework exceptions.	Negative tests for POST/PUT/PATCH/DELETE and configured exceptions.
SEC-003	Passwords shall be stored using a one-way password-hashing function configured for the application; the presentation reports BCrypt.	Inspect security configuration and safe test records.
SEC-004	Session cookies should use Secure in HTTPS deployments, HttpOnly, and an appropriate SameSite policy.	Inspect response headers in deployment-like test environment.
SEC-005	The application shall not expose secrets, passwords, session identifiers, stack traces, or sensitive database details in client-facing errors.	Trigger representative failures and inspect responses/logging.
SEC-006	Configuration secrets should be supplied through environment-specific secure configuration rather than committed source files.	Review repository and deployment configuration.
SEC-007	Dependencies should be reviewed for known vulnerabilities before release.	Run dependency scanning and document findings.
SEC-008	Personal data such as driver and licence details should be accessible only to roles with a legitimate operational need.	Review role matrix and test denied access.

10.3 Privacy Considerations

Driver identity and licence details may be personal information. The application should collect only fields needed for its operational purpose, limit access by role, avoid unnecessary logging, and define retention/deletion expectations. The presentation does not specify a privacy notice, retention schedule, consent model, or regulatory assessment; these are open questions for the project team.

10.4 Threats and Mitigations

- Broken access control: enforce authorization on backend routes/services and test every role.
- CSRF: preserve token handling for state-changing session-authenticated requests.
- Session theft: use secure cookie flags in HTTPS deployments and invalidate sessions on logout.
- Injection or malformed input: validate inputs and use parameterized ORM/database access.
- Information disclosure: sanitize error responses and restrict sensitive logs.
- Dependency vulnerabilities: keep supported framework/library versions and review advisories.
- Misleading location data: label simulated telemetry and estimated routing clearly.

11. Testing and Acceptance Criteria

11.1 Test Strategy

The presentation states that the README describes 79 automated backend unit, module, migration, and end-to-end tests, and that the frontend uses Vitest and React Testing Library with 7 suites and 34 tests. These counts are reported by the presentation and have not been independently verified. The presentation also names smoke checks for health, SPA fallback, authentication, CSRF, data access, and logout. Actual test execution results must be captured from the repository.

11.2 Test Levels

- Unit testing: validate service-level rules, calculations, validation, and state transitions in isolation.
- Integration testing: validate backend modules with persistence and security configuration.
- Migration testing: initialize or upgrade a test database using Flyway migrations.
- Frontend testing: test forms, navigation, role-aware presentation, map/tracking states, and error states.
- End-to-end testing: exercise login through an operational workflow using a representative environment.
- Performance testing: measure agreed API targets under a documented workload.
- Security testing: test role bypass, CSRF, session/logout, unsafe error output, and dependency risks.
- Regression testing: rerun relevant automated suites after changes.

11.3 Test Cases

Test ID	Req. ID	Scenario / input	Expected result	Actual / status
TC-001	FR-001	Submit valid credentials for a configured user	Session is established and authorized view is reached	Not Executed
TC-002	FR-001	Submit invalid credentials	Authentication is rejected with safe feedback	Not Executed
TC-003	FR-003	Viewer attempts an administrative mutation	Backend denies request; data remains unchanged	Not Executed
TC-004	FR-005	Create vehicle with valid required fields	Vehicle is saved and visible in directory	Not Executed
TC-005	FR-005, FR-022	Submit vehicle with missing required field	Validation error; no invalid record is saved	Not Executed
TC-006	FR-007	Create fuel log with valid odometer/fuel values	Record saves and efficiency is calculated with defined units	Not Executed
TC-007	FR-007	Use zero/missing/inconsistent fuel inputs	No divide-by-zero or misleading metric; input is rejected or result withheld	Not Executed
TC-008	FR-009, FR-010	Dispatch a trip with valid resource references	Allowed trip state/assignments are saved	Not Executed
TC-009	FR-011, FR-012	Save shipment and ordered waypoints	Manifest and waypoint order appear consistently	Not Executed
TC-010	FR-013	Run simulated tracking for a configured route	Coordinates progress along waypoints and are labelled simulated	Not Executed
TC-011	FR-015	Disconnect WebSocket during tracking	UI indicates stale/disconnected state or recovers gracefully, as designed	Not Executed
TC-012	FR-017	View route distance/ETA	Values are marked estimates based on straight-line calculation	Not Executed
TC-013	FR-019	Submit state-changing request without CSRF token	Request rejected and no mutation occurs	Not Executed
TC-014	FR-020	Inspect stored password representation in test environment	Password is not plaintext; reported BCrypt configuration is verified	Not Executed
TC-015	FR-021	Initialize clean database	Flyway migrations complete in order and schema validation succeeds	Not Executed
TC-016	NFR-PER-001	Run defined load profile against standard read endpoints	Report percentiles; compare with proposed target	Not Executed
TC-017	SEC-005	Trigger validation and server-side failures	Client response contains no stack trace or secret	Not Executed
TC-018	FR-002	Log out and reuse prior session	Prior session is invalidated according to configured behavior	Not Executed

11.4 Acceptance Criteria

A release candidate should not be accepted solely because screens render. Acceptance should require: core workflows are executable; role restrictions are enforced on the backend; valid records persist; invalid inputs do not create inconsistent data; simulated location is disclosed; estimates are described accurately; automated tests run in a clean documented environment; and unresolved defects/limitations are recorded. Proposed performance targets should be assessed separately and reported with test evidence.

11.5 Test Execution Record

For every executed test, record build/commit identifier, environment, database state, tester, date, actual result, evidence location, and defect reference if failed. Do not change “Not Executed” to “Passed” without a test run and supporting evidence.

12. Requirements Traceability Matrix

The matrix links requirements to presentation-described modules, use cases, and proposed tests. Module labels are logical names from the presentation; they are not asserted to be exact Java package or class names. Replace these labels with source paths after the ZIP is inspected.

Req. ID	Source module / evidence	Use case	Verification
FR-001	Auth module (reported)	UC-01	TC-001, TC-002
FR-002	Auth/session configuration (reported)	UC-01	TC-018
FR-003	Spring Security / role rules (reported)	UC-01, UC-02, UC-03, UC-04, UC-05	TC-003
FR-004	Auth & Admin module (reported)	Administrative workflow	Role matrix tests; not yet enumerated
FR-005	Fleet module (reported)	UC-02	TC-004, TC-005
FR-006	Driver module (reported)	UC-03	Driver validation test; to add after source review
FR-007	Fuel log feature (reported)	UC-05	TC-006, TC-007
FR-008	Maintenance feature (reported)	UC-05	Maintenance due-date tests; to add after source review
FR-009	Drivers & Trips module (reported)	UC-03	TC-008
FR-010	Dispatch workflow (reported)	UC-03	TC-008
FR-011	Shipment module (reported)	UC-03	TC-009
FR-012	Route waypoint feature (reported)	UC-03, UC-04	TC-009
FR-013	SimulatedGpsService (name reported)	UC-04	TC-010
FR-014	Leaflet map UI (reported)	UC-04	Map rendering test; to add
FR-015	STOMP/WebSocket client and backend events (reported)	UC-04	TC-011
FR-016	Tracking / zones / alerts (reported)	UC-04	Geofence/alert rule tests; to add
FR-017	Haversine estimate logic (reported)	UC-04	TC-012
FR-018	Reports feature (reported)	Operational review	Report-specific tests; catalogue needed
FR-019	Spring Security CSRF (reported)	UC-01 / all mutations	TC-013
FR-020	BCrypt password hashing (reported)	UC-01	TC-014
FR-021	Flyway migrations (reported)	Startup/data setup	TC-015
FR-022	Validation logic (proposed)	UC-02, UC-03, UC-05	TC-005, TC-007
FR-023	Error handling (proposed)	Cross-cutting	TC-017
NFR-PER-001	Backend endpoints (exact list unavailable)	Cross-cutting	TC-016
SEC-001	Backend authorization configuration	Cross-cutting	TC-003 plus full role/action matrix
SEC-005	Exception handlers/logging	Cross-cutting	TC-017

12.1 Traceability Gaps

The presentation does not provide exact source file paths, API routes, database migration filenames, test method names, or test execution reports. The traceability links above are therefore feature-level. Before sign-off, inspect the repository and replace logical module labels with verified paths, add all significant requirements, and ensure each implemented requirement maps to at least one test.

13. Limitations, Risks, and Future Enhancements

13.1 Current Limitations Reported

- GPS telemetry is simulated rather than sourced from physical GPS hardware.
- Haversine distance is straight-line distance rather than road-network distance.
- ETA is an estimate and is not traffic-aware.
- Production deployment requires environment-specific setup.

13.2 Risks and Mitigation

Risk	Potential impact	Mitigation
Simulation mistaken for real tracking	Incorrect operational decisions	Persistent “SIMULATED DATA” labels and explanatory text
Straight-line distance mistaken for road distance	Unrealistic distance/ETA expectations	Label estimates; use a road-routing provider only as a separately scoped enhancement
Role misconfiguration	Unauthorized data access or mutation	Backend authorization matrix and negative security tests
Schema migration failure	Startup or data integrity problems	Test clean installs and upgrades against representative database versions
Map tile/network outage	Map may not render fully	Provide non-map trip/waypoint details and clear map error state
Unverified performance	Slow experience under load	Run repeatable workload tests before making performance claims
Insufficient backup strategy	Potential data loss	Define environment-specific backups and recovery drills for deployed environments

13.3 Future Enhancements — Not Current Requirements

- Integrate physical GPS/telematics devices through a documented adapter and authenticated ingestion pipeline.
- Use road-network routing for realistic route distance and ETA; consider traffic-aware estimates only with an appropriate data provider.
- Add configurable notification channels for operational alerts if needed.
- Introduce audit trails and exportable reports if not already implemented.
- Add deployment observability, metrics, structured logs, health checks, and alerting appropriate to the hosting environment.
- Define automated backup/restore procedures and disaster-recovery objectives.
- Improve accessibility and responsive behavior based on evaluation findings.

13.4 Out of Scope Unless Confirmed

Real vehicle hardware integration, traffic-aware routing, predictive maintenance, machine-learning-based ETA prediction, production multi-tenant hosting, mobile-native applications, and third-party carrier integrations are not established by the supplied presentation and should not be represented as implemented features.

14. Deployment and Maintenance

14.1 Environment Setup

The presentation lists Maven and npm, and states that README.md documents the commands `./mvnw test`, `npm test -- --run`, and `npm run build`. These commands should be verified in the source repository. Exact startup commands, environment variable names, Docker Compose services, database credentials, ports, and migration behavior are not visible in the presentation.

14.2 Prerequisites

- Java 21-compatible runtime and build environment for the backend.
- Node.js/npm versions compatible with the frontend lockfile and scripts.
- MySQL 8.4-compatible database service.
- Browser with JavaScript and WebSocket support.
- Network access to configured map tiles when using the map view.
- Environment-specific configuration for database connection and application secrets; exact keys must be read from project configuration.

14.3 Build and Verification Commands Reported

Command	Purpose stated by presentation	Caveat
`./mvnw test`	Run backend automated tests	Verify wrapper and test configuration in repository
`npm test -- --run`	Run frontend tests	Verify package.json script and test runner
`npm run build`	Build frontend production assets	Verify package.json and build output
Backend startup command	Not provided in supplied presentation	Obtain exact command from README/build files
Database startup command	Not provided in supplied presentation	Inspect Docker Compose/configuration files

14.4 Startup, Shutdown, and Configuration

The final deployment instructions should define database startup, schema migration, backend startup, frontend serving/proxy configuration, health checks, shutdown order, and recovery steps. Secrets should not be committed to source control. The presentation reports a persistent Docker volume for local MySQL storage, but the volume name and configuration are not supplied. No exact production topology can be inferred.

14.5 Maintenance

- Keep Java/Spring, frontend dependencies, database image, and libraries within supported versions.
- Review dependency vulnerability reports before releases.
- Run backend and frontend automated tests after changes.
- Back up persistent database data in deployed environments and periodically test restoration.
- Review role assignments and access when team responsibilities change.
- Document migration rollback/recovery strategy; do not assume migrations can always be safely reversed.
- Update this SRS and traceability matrix when requirements or implementation change.

14.6 Troubleshooting Items to Document

The repository-specific guide should cover database connection failures, failed migrations, port conflicts, authentication/session problems, CSRF failures, frontend-to-backend proxy/CORS issues if applicable, WebSocket connection failures, unavailable map tiles, and missing environment configuration. Exact remedies should be added only after inspecting the actual configuration and README.

15. Conclusion and Appendices

15.1 Conclusion

FleetX is presented as a unified fleet and logistics web application combining fleet and driver records, trip dispatch, shipment manifests, fuel and maintenance information, map-based route visibility, simulated tracking, alerts, and reports. The stated implementation uses a React/TypeScript frontend, a Java/Spring Boot modular-monolith backend, and MySQL persistence with JPA/Hibernate and Flyway. The SRS sets out functional expectations, quality targets, interfaces, use cases, data considerations, security requirements, tests, and traceability. Approval should follow source-code verification and execution of the proposed tests.

Appendix A: Project Directory Structure

The actual directory tree is not available because the uploaded artifact is a presentation rather than a project ZIP. Insert the output of a repository tree command here after uploading the source archive. Do not infer directory or package names from the technology stack.

Appendix B: Important Classes and Responsibilities

The presentation explicitly names `SimulatedGpsService`. Other Java class names, interfaces, controllers, DTOs, entities, repositories, configuration classes, and methods are not specified. Populate this appendix from source inspection, including each verified file path, responsibility, and linked requirement ID.

Appendix C: API Reference

Not specified in the supplied project presentation. Extract actual controller mappings, request/response DTOs, validation rules, authentication requirements, and error responses from the source repository before publishing an API catalogue.

Appendix D: Glossary

Term	Definition
Control tower	Operational view for monitoring fleet status, route progress, and supported alerts.
Geofence	A logical geographic boundary used to detect or represent entry/exit conditions; exact rules must be verified.
Haversine calculation	A spherical-earth calculation for straight-line distance between coordinates; it is not road distance.
Modular monolith	One deployable application organized into modules rather than separately deployed microservices.
Simulated GPS	Generated location data used to demonstrate tracking without proving the physical location of a real vehicle.
Traceability	The relationship between requirements, implementation evidence, use cases, and tests.

Appendix E: Open Questions Before Final Approval

- What are the exact backend endpoint paths and payload schemas?
- What are the verified Java package/class names and frontend routes?
- What is the actual MySQL schema and migration sequence?
- What are the precise role permissions for each operation?
- What are the supported trip states and valid state transitions?
- What are the exact geofence and alert triggering rules?
- Are the test counts reported in the presentation current, and what is the last test run result?
- What are the required runtime environment variables and deployment steps?
- What data retention, backup, and privacy rules are intended?

Appendix F: References

- Project source basis: `FleetX_Project_Presentation_Final.pptx`, supplied by the user, presentation slides covering project overview, architecture, stack, workflows, security, testing, and limitations.
- Java Platform documentation: https://docs.oracle.com/en/java/
- Spring Boot reference documentation: https://docs.spring.io/spring-boot/
- Spring Security reference documentation: https://docs.spring.io/spring-security/reference/
- MySQL documentation: https://dev.mysql.com/doc/
- Hibernate ORM documentation: https://hibernate.org/orm/documentation/
- Flyway documentation: https://documentation.red-gate.com/flyway

• React documentation: https://react.dev/

• Leaflet documentation: https://leafletjs.com/reference.html

• OpenStreetMap tile usage policy: https://operations.osmfoundation.org/policies/tiles/

• Mermaid diagram syntax: https://mermaid.js.org/